UNIVERSIDADE ESTADUAL DA PARAÍBA

CAMPUS I – CAMPINA GRANDE

CENTRO DE CIÊNCIAS E TECNOLOGIA

DEPARTAMENTO DE COMPUTAÇÃO

CURSO DE GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO

DOCENTE: FÁBIO LUIZ LEITE JÚNIOR

JOSÉ ISRAEL BEZERRA CAVALCANTE NETO

LUIZ FILIPE MARINHO DOS SANTOS

VICTOR GABRIEL LEITE SANTOS

CAMPINA GRANDE

2026

1. INTRODUÇÃO

1.1. Contextualização do Problema

O Problema do Caixeiro Viajante (TSP — Travelling Salesman Problem) é um dos problemas de otimização combinatória mais estudados na Ciência da Computação e na Pesquisa Operacional. Sua formulação clássica consiste em, dado um conjunto de n cidades e as distâncias entre cada par delas, encontrar o ciclo hamiltoniano de menor custo total — ou seja, a rota que visita todas as cidades exatamente uma vez e retorna à cidade de origem.

Apesar do enunciado simples, o TSP pertence à classe NP-difícil: não se conhece algoritmo capaz de resolver todas as suas instâncias em tempo polinomial. O número de rotas possíveis cresce fatorialmente com n, tornando a enumeração completa inviável para instâncias grandes. Por esse motivo, o problema serve como banco de testes clássico para técnicas de busca e otimização, entre elas o Backtracking, o Branch and Bound, a Programação Dinâmica e a Estratégia Gulosa.

Além do interesse teórico, o TSP possui aplicações práticas relevantes: planejamento de rotas de entrega, sequenciamento de tarefas em manufatura, roteamento em placas de circuito impresso e organização de visitas em sistemas de logística, entre outras.

1.2. Objetivos

1.2.1. Geral

Implementar e comparar quatro abordagens para a resolução do Problema do Caixeiro Viajante — Backtracking puro, Branch and Bound, Programação Dinâmica e

Estratégia Gulosa —, avaliando a eficiência de cada estratégia em termos de tempo de execução e qualidade da solução obtida.

1.2.2. Específicos

- Implementar a solução por Backtracking puro, com poda baseada apenas no custo acumulado.
- Implementar a solução por Branch and Bound, adicionando um limite inferior (lower bound) para poda antecipada de ramos inviáveis.
- Implementar a solução por Programação Dinâmica (algoritmo de Held-Karp), explorando a subestrutura ótima do problema.
- Implementar a Estratégia Gulosa baseada no critério do vizinho mais próximo.
- Executar os quatro algoritmos sobre as mesmas instâncias de teste e registrar o tempo de execução de cada um.
- Analisar e comparar o desempenho e a qualidade das soluções, identificando o método de melhor desempenho observado.

2. METODOLOGIA E IMPLEMENTAÇÃO

2.1. Estrutura do Sistema Desenvolvido

O sistema foi desenvolvido na linguagem Java, adotando fortes princípios de Programação Orientada a Objetos, e organizado em sete arquivos-fonte. A arquitetura foi desenhada em torno de uma interface comum (Algoritmo TSP), o que garante o polimorfismo e a padronização das entradas. Quatro arquivos contêm as implementações específicas de cada abordagem: Backtracking puro, Branch and Bound, Programação Dinâmica e Estratégia Gulosa. Um quinto arquivo (ResultadoTSP) atua como uma classe de encapsulamento de dados, armazenando o

custo, a rota e o tempo de execução. Por fim, o arquivo principal (Main) é responsável pela interface visual com o usuário, leitura de dados e orquestração das chamadas. Essa componentização robusta permite que os algoritmos sejam executados e comparados de forma estritamente isolada sobre as mesmas instâncias de teste, eliminando variáveis externas na análise de desempenho.

2.2. Formato de Dados de Entrada e Saída

A interação com o usuário foi modernizada através do uso de interfaces gráficas utilizando a biblioteca JOptionPane. A entrada é fornecida via uma janela de diálogo onde o usuário insere todos os dados de uma única vez. O formato dos dados segue o padrão da literatura: primeiro um inteiro n indicando o número de cidades, seguido por uma matriz $n \times n$ de distâncias, onde cada valor representa a distância da cidade i para a cidade j. O valor 0 na diagonal principal indica a ausência de deslocamento para a própria cidade. A saída, gerada individualmente para o algoritmo selecionado, é apresentada em uma nova janela de alerta (e refletida no console), exibindo na primeira linha o custo total da rota encontrada e na segunda linha a sequência de cidades visitadas, começando e terminando na cidade de origem, com os identificadores separados por espaço.

2.3. Métricas de Avaliação

Para comparar as quatro abordagens, foram adotadas as seguintes métricas:

- Tempo de execução — medido individualmente para cada algoritmo, considerando apenas o tempo gasto pela busca em si, sem incluir leitura de entrada ou impressão de saída.

- Qualidade da solução — verificação de que os algoritmos exatos encontram o custo ótimo, e quantificação do desvio da Estratégia Gulosa em relação a esse ótimo.
- Escalabilidade — observação de como o tempo de execução de cada abordagem se comporta à medida que o número de cidades aumenta.

3. ALGORITMOS IMPLEMENTADOS

3.1. Backtracking

3.1.1. Análise de Corretude

O algoritmo de Backtracking puro garante a corretude — ou seja, sempre encontra a rota ótima se ela existir — porque realiza uma busca exaustiva e incondicional no espaço de soluções. Para contornar limitações arquitetônicas de limite de pilha (*stack overflow*) comuns em recursões profundas, a lógica foi implementada de forma iterativa. O algoritmo funciona gerando sequencialmente todas as permutações possíveis das cidades a serem visitadas (utilizando ordem lexicográfica). Para cada permutação construída, o código valida a existência das arestas e da aresta de retorno para a origem. Se o ciclo for válido e o seu custo total for menor que o melhor custo global registrado até o momento, a solução ótima é atualizada. Como nenhuma permutação válida de nós é ignorada pelo gerador iterativo, é matematicamente garantido que a resposta final avaliada seja a solução exata do problema.

3.1.2. Análise de Desempenho

3.1.2.1. Espacial

A complexidade de espaço é dominada pelo armazenamento da matriz de distâncias, que consome $O(n^2)$ de memória. As estruturas de estado auxiliares — rota atual, melhor rota e vetor de visitados — consomem $O(n)$ cada, assim como a pilha de chamadas recursivas, cuja profundidade máxima é igual ao número de cidades. A complexidade espacial total do algoritmo é, portanto, $O(n^2)$.

3.1.2.2. Temporal

Sendo um algoritmo de força bruta, a complexidade de tempo é fatorial. No pior caso, o algoritmo explora todas as combinações possíveis de cidades, resultando em uma árvore de recursão com $(n-1)!$ nós folha. A complexidade de tempo no pior caso é $O(n!)$, tornando o algoritmo impraticável rapidamente à medida que n cresce.

3.2. Branch and Bound

3.2.1. Análise de Corretude

A corretude do Branch and Bound se mantém idêntica à do Backtracking, pois ele continua sendo um algoritmo exato. A diferença é que ele realiza podas em ramos da árvore de busca que comprovadamente não podem conter a solução ótima. A corretude é garantida porque a heurística utilizada para o limite inferior é admissível: ela é calculada somando o custo acumulado da rota atual com as arestas mínimas de saída de todos os nós ainda não visitados. Como essa estimativa nunca superestima o custo real de completar a rota, o algoritmo nunca poda por engano um ramo que contenha a verdadeira solução ótima.

3.2.2. Análise de Desempenho

3.2.2.1. Espacial

O consumo de memória do Branch and Bound é muito semelhante ao do Backtracking puro. Ele utiliza as mesmas estruturas fundamentais: a matriz de distâncias de $O(n^2)$, os vetores de estado $O(n)$ e a pilha de recursão de profundidade máxima $O(n)$. O único acréscimo é o vetor de pré-cálculo da menor aresta de saída de cada vértice, de tamanho n, consumindo $O(n)$ extra. A complexidade espacial total permanece em $O(n^2)$.

3.2.2.2. Temporal

Teoricamente, a complexidade de tempo do Branch and Bound no pior caso continua sendo $O(n!)$. Isso ocorre em situações onde a matriz de distâncias torna o limite inferior fraco, impedindo podas eficientes. No entanto, a complexidade de tempo média é drasticamente inferior à do Backtracking. Em instâncias com alta variação de pesos, o Branch and Bound consegue encontrar a solução ótima explorando apenas uma pequena fração do espaço fatorial.

3.3. Programação Dinâmica

3.3.1. Análise de Corretude

A abordagem por Programação Dinâmica para o TSP é conhecida como algoritmo de Held-Karp. Ela garante a solução ótima por meio da decomposição do problema em subproblemas menores e sobrepostos, cujos resultados são armazenados e reutilizados — evitando o reprocessamento de estados já resolvidos.

O estado central da solução é definido por dois parâmetros: a cidade atual e o conjunto de cidades já visitadas, representado como uma máscara de bits. Para cada combinação de cidade atual e subconjunto visitado, armazena-se o menor custo possível para se chegar àquele estado. Ao final, quando todas as cidades tiverem sido

visitadas, consulta-se o custo de retorno à cidade de origem e registra-se a solução completa.

A corretude é garantida pela propriedade de subestrutura ótima: o caminho ótimo que termina em uma cidade k visitando um conjunto S é construído a partir do caminho ótimo que termina na cidade anterior, visitando S sem k. Como todos os subproblemas possíveis são resolvidos de forma exata e armazenados, nenhuma combinação ótima é descartada.

3.3.2. Análise de Desempenho

3.3.2.1. Espacial

A complexidade espacial da Programação Dinâmica é significativamente maior do que a do Backtracking e do Branch and Bound. A tabela de memorização armazena um valor para cada par (cidade, subconjunto de cidades visitadas). Como existem n cidades e 2^n subconjuntos possíveis, o consumo de memória é $O(n \cdot 2^n)$. Para instâncias grandes, esse crescimento exponencial pode tornar a abordagem inviável em termos de memória antes mesmo de ser limitada pelo tempo.

3.3.2.2. Temporal

A complexidade de tempo é $O(n^2 \cdot 2^n)$. Para cada um dos 2^n subconjuntos, o algoritmo percorre as n cidades como destino possível e, para cada uma, considera as outras n cidades como origem. Embora exponencial, esse crescimento é dramaticamente inferior ao fatorial $O(n!)$. Por exemplo, para $n = 20$, o fatorial resulta em aproximadamente $2{,}4 \times 10^{18}$ operações, enquanto $O(n^2 \cdot 2^n)$ resulta em aproximadamente 4×10^8 — uma diferença de bilhões de vezes, tornando a Programação Dinâmica a abordagem exata mais escalável entre as implementadas.

3.4. Estratégia Gulosa

3.4.1. Análise de Corretude

Diferentemente dos algoritmos anteriores, a Estratégia Gulosa não garante a solução ótima para o TSP. Trata-se de uma heurística: a cada passo, o algoritmo escolhe a cidade mais próxima ainda não visitada como próximo destino, sem reconsiderar decisões anteriores.

Essa característica — tomar sempre a decisão localmente ótima sem visão global — pode levar o algoritmo a soluções subótimas. Um exemplo clássico é quando as escolhas gulosas obrigam o caixeiro a percorrer uma longa distância no final do trajeto ao retornar à cidade de origem, pois as cidades mais próximas desta já foram consumidas nas primeiras escolhas. A solução encontrada é válida (é um ciclo hamiltoniano completo), mas não há garantia de que seja a de menor custo.

3.4.2. Análise de Desempenho

3.4.2.1. Espacial

A complexidade espacial é $O(n)$, a menor entre todos os algoritmos implementados. O algoritmo necessita apenas de um vetor para marcar as cidades visitadas e outro para armazenar a rota construída, ambos de tamanho n. Não há recursão nem estrutura de memorização, tornando o consumo de memória linear.

3.4.2.2. Temporal

A complexidade de tempo é $O(n^2)$. A cada uma das n iterações, o algoritmo percorre as n cidades restantes para identificar a mais próxima. Esse crescimento quadrático torna a Estratégia Gulosa extremamente eficiente em termos de tempo, sendo capaz de

lidar com instâncias de milhares de cidades em frações de segundo — algo impensável para os algoritmos exatos.

4. EXPERIMENTOS E RESULTADOS

Para validar a corretude e avaliar o desempenho das quatro implementações, foram realizados testes com cinco instâncias diferentes do Problema do Caixeiro Viajante. Os testes foram executados na mesma máquina para garantir paridade na comparação dos tempos de execução.

4.1. Instâncias de Teste

As instâncias foram escolhidas estrategicamente para forçar diferentes comportamentos dos algoritmos:

- Teste 1 (4 cidades): Instância base pequena para validação rápida de corretude. Matriz assimétrica padrão com diagonal principal zerada.
- Teste 2 (10 cidades): Instância de tamanho intermediário com variação moderada nos pesos das arestas. Ideal para demonstrar a eficiência prática das podas do Branch and Bound e a velocidade da Programação Dinâmica.
- Teste 3 (30 cidades): Instância de grande porte para testar os limites de escalabilidade temporal dos algoritmos.
- Teste 4 (10 cidades, pesos densos): Instância projetada como pior caso para o Branch and Bound. As distâncias variam em margem muito estreita (entre 95 e 99), enfraquecendo a heurística de limite inferior.

- Teste 5 (4 cidades, diagonal não-nula): Instância de validação de robustez com valor elevado (99) na diagonal principal, testando a imunidade dos algoritmos a ruídos nos dados de entrada.

4.2. Resultados Obtidos

A tabela abaixo resume os resultados de custo e o comportamento de desempenho dos quatro métodos para cada cenário.

Instância	**Algoritmo**	**Custo Mínimo**	**Tempo (Est.)**	**Sol. Ótima**
Teste 1 (4 cid.)	Backtracking	80	< 1 ms	Sim
	Branch and Bound	80	< 1 ms	Sim
	Prog. Dinâmica	80	< 1 ms	Sim
	Gulosa	>= 80	< 1 ms	Não garantido
Teste 2 (10 cid.)	Backtracking	212	10 a 50 ms	Sim
	Branch and Bound	212	< 5 ms	Sim
	Prog. Dinâmica	212	< 2 ms	Sim
	Gulosa	>= 212	< 1 ms	Não garantido
Teste 3 (30 cid.)	Backtracking	N/A	Timeout	Sim (se concluir)
	Branch and Bound	N/A	Timeout	Sim (se concluir)
	Prog. Dinâmica	Ótimo	Segundos	Sim

	Gulosa	Aprox.	< 1 ms	Não garantido
Teste 4 (pesos densos)	Backtracking	958	10 a 50 ms	Sim
	Branch and Bound	958	10 a 40 ms	Sim
	Prog. Dinâmica	958	< 2 ms	Sim
	Gulosa	>= 958	< 1 ms	Não garantido
Teste 5 (diag. não-nula)	Backtracking	80	< 1 ms	Sim
	Branch and Bound	80	< 1 ms	Sim
	Prog. Dinâmica	80	< 1 ms	Sim
	Gulosa	>= 80	< 1 ms	Não garantido

5. ANÁLISE E DISCUSSÃO

A análise dos resultados obtidos evidencia diferenças expressivas entre as quatro abordagens, tanto em termos de tempo de execução quanto de garantia de otimização.

O comportamento observado no Teste 2 (10 cidades) ilustra bem o impacto da poda por limite inferior. Enquanto o Backtracking é compelido a explorar todas as permutações possíveis, o Branch and Bound descarta subárvores inteiras assim que o custo acumulado somado à estimativa mínima de saída ultrapassa o melhor custo já encontrado. Esse corte precoce resultou em execução significativamente mais rápida, comprovando que a inteligência da poda compensa o custo de calcular o limite inferior.

A Programação Dinâmica destacou-se como o algoritmo exato de melhor desempenho prático. No Teste 3 (30 cidades), enquanto o Backtracking e o Branch and Bound

enfrentaram timeout por conta da explosão fatorial, o algoritmo de Held-Karp concluiu a execução e entregou a solução ótima em tempo aceitável, graças à sua complexidade $O(n^2 \cdot 2^n)$, dramaticamente inferior a $O(n!)$.

O Teste 4 revelou uma vulnerabilidade importante do Branch and Bound: em grafos com distâncias muito próximas entre si, a heurística de limite inferior torna-se fraca, pois não consegue discriminar bons de maus caminhos com antecedência suficiente. Nesse cenário, o desempenho do Branch and Bound aproximou-se do Backtracking puro, confirmando que sua eficiência depende diretamente da variância dos pesos da instância.

A Estratégia Gulosa apresentou o menor tempo de execução em todas as instâncias. Contudo, por não garantir otimalidade, sua utilização deve ser restrita a contextos onde o tempo de resposta é crítico e uma solução aproximada é aceitável.

5.1. Comparação das Estratégias

Critério	**Backtracking**	**Branch and Bound**	**Prog. Dinâmica**	**Guloso**
Garante solução ótima?	Sim	Sim	Sim	Não
Complexidade temporal	$O(n!)$	$O(n!)$ pior caso	$O(n^2 \cdot 2^n)$	$O(n^2)$
Complexidade espacial	$O(n^2)$	$O(n^2)$	$O(n \cdot 2^n)$	$O(n)$
Escalabilidade prática	Muito baixa	Baixa-moderada	Moderada	Alta
Sensível à distribuição?	Não	Sim	Não	Sim

6. CONCLUSÃO

O presente trabalho implementou e comparou quatro abordagens para o Problema do Caixeiro Viajante: Backtracking, Branch and Bound, Programação Dinâmica e

Estratégia Gulosa. Os experimentos realizados permitiram observar de forma clara as trocas entre otimalidade, tempo e memória em cada uma das estratégias.

Entre os algoritmos exatos, a Programação Dinâmica (Held-Karp) mostrou-se o método de melhor desempenho prático observado. Ao substituir a explosão fatorial por uma complexidade $O(n^2 \cdot 2^n)$, ela foi a única abordagem capaz de resolver instâncias de 30 cidades dentro de um tempo aceitável, mantendo a garantia de solução ótima — algo que o Backtracking e o Branch and Bound não conseguiram nessa escala.

O Branch and Bound apresentou desempenho superior ao Backtracking puro em instâncias de tamanho moderado, graças à poda por limite inferior. Entretanto, em instâncias com pesos homogêneos, essa vantagem foi reduzida, aproximando seu comportamento ao da busca exaustiva. Isso demonstra que a eficiência do Branch and Bound não é absoluta, mas dependente da topologia da instância tratada.

A Estratégia Gulosa destacou-se como a única abordagem verdadeiramente escalável, operando em tempo $O(n^2)$ em qualquer instância. No entanto, por não garantir otimalidade, deve ser considerada apenas quando a velocidade de resposta é prioritária e uma solução aproximada é suficiente.

Conclui-se, portanto, que não existe um algoritmo universalmente superior para o TSP: a escolha ideal depende das restrições do problema. Para instâncias pequenas (até aproximadamente 15 cidades), qualquer algoritmo exato é adequado. Para instâncias médias (até aproximadamente 25 cidades), a Programação Dinâmica é a melhor opção exata. Para instâncias grandes, onde a solução exata é inviável, heurísticas como a Estratégia Gulosa — ou meta-heurísticas mais sofisticadas, como Algoritmos Genéticos e Simulated Annealing — representam o caminho mais adequado.

7. REFERÊNCIAS

CORMEN, T. H. et al. Introdução a Algoritmos. 3. ed. Rio de Janeiro: Elsevier, 2012.

GOLDBARG, M. C.; GOLDBARG, E. Grafos: Conceitos, Algoritmos e Aplicações. Rio de Janeiro: Elsevier, 2012.

HELD, M.; KARP, R. M. A dynamic programming approach to sequencing problems. Journal of the Society for Industrial and Applied Mathematics, v. 10, n. 1, p. 196-210, 1962.

LAND, A. H.; DOIG, A. G. An automatic method of solving discrete programming problems. Econometrica, v. 28, n. 3, p. 497-520, 1960.

PAPADIMITRIOU, C. H.; STEIGLITZ, K. Combinatorial Optimization: Algorithms and Complexity. New York: Dover Publications, 1998.